DATA PIPELINE

데이터 수집·활용 절차

출처 → 수집 → 전송 → 가공 → 저장 → 활용 | white-eyes 데이터 흐름 한눈에 보기

01
출처

· 드론·CCTV 영상 (LIVE RTSP / DUMMY MP4)

· 행정구역 경계 GeoJSON

· 모델 학습: VisDrone

02
수집

· 분석 서버 Flask (Python)

· OpenCV → YOLOv8m 탐지

· PointTracker 추적·체류 계산

03
전송

· 1초마다 POST /internal/v1/results

· JSON: 인원·in/out·체류

· 내부 토큰 인증 · 실패 시 5분 재전송

04
가공

· Spring 관리자 앱

· 유효성 확인 · 구역별 누적

· 1분 → 시간 → 일 집계

05
저장

· MySQL: 통계·이벤트

· 메모리: 현재 인원

· 파일: MP4 · JSON.gz

06
활용

· 지도 · 그래프 · 대시보드

· 엑셀 · 이벤트 알림

· 다시보기 (영상 + 박스)

저장·전달 경로 3가지 · 관리자 앱은 분석 서버에 캠 시작·정지를 지시(제어 흐름)

통계 → DB (MySQL)
1초 결과 → 1분 통계 → 시간별 → 일별 → MySQL 저장 → 지도·그래프·엑셀

현재 인원 → 메모리
최신 초 인원 → admin 메모리 → 5초 주기 SSE(now) → 관리자 화면 (15초 넘으면 1분 평균)

영상·좌표 → 파일
MP4 + JSON.gz → 공유 폴더(NFS) → nginx 전달 → 관리자 다시보기

영상 한 장이 '1초 결과 → 1분 통계'로 바뀌어 지도·그래프·엑셀·알림으로 나갑니다

※ 근거: white-eyes-docs 명세 · white-eyes-spring / white-eyes-flask(dev) 구조 (Notion 정리 2026-10-29). 문서상 설계 흐름이며 실제 운영 수집을 검증한 결과가 아님

※ 센 인원은 추정치(VisDrone 사진 120장 기준 실제의 약 60~65%, 상관계수 0.93) · 입력 영상·모델·행정구역은 출처를 각각 따로 기록

DATA PROCESSING & STORAGE

가공·저장 — 1초 결과가 통계로

관리자 앱(Spring admin)이 1초 결과를 모아 1분 → 시간 → 일 통계로 만들고 저장합니다

1초 결과 수신

내부 API로 수신 후 204 응답 (FNC-A-024)

분별 메모리 누적

구역·분 단위로 합산, 같은 초 중복은 마지막 값만 사용

1분 통계

분 끝 + 5초 뒤 마감 → zone_stat_minute (90일 보관)

시간별 통계

매시 01분 30초 · 1분 통계를 가중 평균 → zone_stat_hourly

일별 통계

매일 00:05 · 시간별 통계 합산 → zone_stat_daily

수신할 때 걸러내는 것

- 삭제·정지·없는 캠의 결과는 제외
- 이미 마감된 분에 늦게 도착한 결과 제외
- 끊긴 구간은 0명이 아닌 '빈 구간'
- 수신 표본 수(sample_sec)를 함께 저장

1분 통계 계산

- 인원 평균·최대·최소, 방향별 in/out 합
- 밀집도 = 평균 인원 ÷ 구역 면적(명/㎡)
- 체류 시간은 합과 개수로 저장
- 예) 100㎡·평균 80명 → 0.8명/㎡ (설명용 수치)

이벤트 판정 (1분마다)

- 혼잡 CONGESTION: 주의 0.7 · 위험 1.5 명/㎡
- 급증 SURGE: 10분 전보다 +50% 이상
- 캠 이상 CAM_FAULT: 오류 또는 결과 없음
- 같은 구역·종류의 열린 이벤트는 갱신

저장소 구성과 보관 정책 (명세 기준 값)

메모리 (admin)

- 분별 누적기 · 최신 초 인원
- 현재 인원은 15초 이내 값만 사용, 5초마다 갱신

MySQL 8.x (Flyway)

- 통계 3종·이벤트·계정·작업 기록·녹화 정보
- 1분 통계 90일·녹화 기록 30일 삭제 · DB 백업 7일

공유 폴더 (NFS)

- 라이브 MP4·박스 좌표 JSON.gz·더미 원본
- 라이브 파일은 32일째부터 정리

분석 서버 로컬

- 더미 분석 캐시 · 녹화 중 임시 파일
- 쓰기 실패 시 로컬에 두고 다음 분에 다시 이전

※ 보관 일수와 예약 시각은 명세·설정값이며 실제 운영 정책·작업 성공을 확인한 결과가 아님 · 기준값(THRESHOLD_* 등)은 개발 중 실제 영상으로 조정하는 값 · 모든 시각은 KST
※ 통계 저장 시 (zone_id, stat_at) 기준으로 덮어써서 같은 분을 다시 처리해도 중복되지 않음 · 탐지 박스 좌표는 DB가 아닌 파일에 보관

DATA UTILIZATION

활용 — 누가 무엇으로 쓰나

저장된 통계는 지도·그래프·엑셀·알림·다시보기로 나가고, 역할에 따라 볼 수 있는 범위가 다릅니다

사용자
사용자 앱 · 통계만 사용

- 지도: 시 → 구 → 동 → 구역, 혼잡 단계 색
- 그래프 · 대시보드 · 시간대 히트맵
- 구역 간 이동 · 엑셀 다운로드
- 영상은 볼 수 없음

관리자
관리자 앱 · 통계 + 영상 + 이벤트

- 사용자 기능 전체
- 실시간 화면 · 캠 카드 미리보기 · 다시보기
- 이벤트 확인·조치, 캠 가동·정지
- 구역·캠 관리

최고 관리자
관리자 전체 + 작업 기록

- 관리자 기능 전체
- 관리자 계정 관리
- 작업 기록 조회 (다운로드·이벤트 처리 등)

저장된 데이터를 내보내는 4가지 방법

지도·그래프·대시보드
- 그래프 점을 눌러 연→월→일→시간→분
- SSE로 5초마다 확인, 새로고침 없이 갱신
- 시연용 데이터는 '시연용 포함' 표시

엑셀 다운로드
- 구역·지역·전체 × 기간·단위·항목 선택
- Apache POI SXSSF 스트리밍으로 작성
- 100만 행을 넘으면 거절 · 작업 기록 저장

이벤트 알림·조치
- 새 이벤트를 관리자 화면에 실시간 알림
- 미확인 → 확인 → 조치완료
- 이벤트를 누르면 그 시점 다시보기로 이동

다시보기 (영상 + 박스)
- 그래프 지점을 눌러 그 시각의 녹화 요청
- Spring은 권한만 확인, nginx가 파일 전달
- 영상 위에 탐지 박스를 겹쳐 표시

최종 목적(설계 목표): 누적 데이터로 시간대별·계절별 구역 활성화 정도를 보고 상권 분석·도시 설계에 활용

※ 인원·유입/유출·체류 시간은 탐지·추적 기반 추정값이며 촬영 방향·가림에 따라 오차가 있음 · 시연용 과거 통계는 is_demo로 구분하며 실제 장기 관측 결과가 아님
※ 근거: white-eyes-docs features/*.md · white-eyes-spring admin(stat·media) · 공개 가능한 영상만 사용 (캡처 미포함)